

Can a Catgirl Make Compiler Errors Better?

Proposed Study of Persona-Based AI Compiler Feedback

Study Protocol and Literature Map

Designed for a rapid HCI + AI study with a programming-languages context

Working draft - September 2026

Core idea

Compiler diagnostics are a human-computer interface. This study tests whether adding a deliberately social, friendly persona to an otherwise equivalent AI-generated explanation improves the user experience of compiler feedback. Dr. Meowra is the experimental persona; the scientific construct is persona-based or character-mediated programming feedback.

Not an IRB application. Obtain the appropriate institutional ethics/IRB determination before collecting data intended for publication.

1. Study in One Page

Research problem. Programming error messages are often difficult to interpret, and decades of work have treated diagnostic design as a human-factors problem. Recent LLM systems can generate context-aware explanations, but the existence of a better explanation does not answer a different HCI question: how should that explanation communicate with the programmer?

Primary research question. How does the presentation style of compiler feedback - raw diagnostic, neutral AI explanation, or persona-based AI explanation - affect the user experience of receiving and interpreting compiler errors?

Primary outcome. User experience, measured principally with the UEQ-S and a small set of study-specific 7-point agreement items. Comprehension is retained only as a lightweight secondary check, not as the central contribution.

Condition	What the participant sees	Scientific role
A. Raw Diagnostic	Compiler output with no additional explanation.	Technical baseline.
B. Neutral AI Explanation	A concise, friendly-but-impersonal explanation generated from the same source information used for the persona condition.	Controls for the benefit of added explanation.
C. Dr. Meowra Persona	The same technical content and guidance, expressed through a named, warm, socially framed catgirl programming assistant.	Tests the incremental effect of persona/social framing.

Recommended scope. A within-subject Unity study with six short compiler-error scenarios (two per condition), counterbalanced condition order, a UEQ-S evaluation after each condition, four targeted 7-point items after each condition, selected Agent Persona Instrument subscales once for Dr. Meowra, one final forced preference, and one open-ended explanation.

The key contrast

Raw vs. Neutral asks whether explanatory rewriting improves UX. Neutral vs. Dr. Meowra asks the novel question: when technical information is held approximately constant, does social/persona framing change UX?

2. Why This Is a Defensible HCI + AI + PL Paper

Compiler diagnostics have long been framed as an HCI problem. Traver (2010) argued that compiler errors form an interface between the compiler and the programmer and explicitly imagined affective computing in compiler construction: future tools could vary how messages are stated and reassure a frustrated programmer. That historical observation is unusually well matched to modern generative AI, which can now produce contextual natural-language explanations on demand.

The field has already studied whether messages can be clearer. Work on enhanced error messages, readability, explanatory structure, and visual presentation shows both positive and null results. This mixed literature is useful: it cautions against assuming that simply making a message

longer or AI-generated will improve outcomes (Becker, 2016; Denny et al., 2014, 2020; Dong & Khandwala, 2019; Barik et al., 2018).

The AI-assisted debugging literature makes the proposed question timely rather than redundant. Recent systems such as `dcc -help` and DCC Sidekick generate contextual compiler explanations with LLMs, while Santos and Becker (2024) found that GPT-4 explanations did not consistently improve time-to-fix relative to alternatives. The opening for this study is therefore not “can an LLM explain an error?” but “what is the UX effect of giving that explanation a social persona?”

HCI provides the theoretical bridge. The Computers Are Social Actors tradition shows that people can respond socially to computers; pedagogical-agent research identifies a “persona effect”; social-agency work suggests that conversational and agent-based cues can change user experience; and recent LLM work shows that anthropomorphic cues can affect how users perceive an AI system (Nass et al., 1994; Lester et al., 1997; Moreno et al., 2001; Cohn et al., 2024).

Historical hook for the introduction

In 2010, affective computing was proposed as a possible future direction for compiler diagnostics. Sixteen years later, LLMs make that idea practical. Dr. Meowra is a deliberately strong test case of what happens when compiler feedback stops behaving like a log message and starts behaving like a social interface.

Primary RQ

RQ1. How does compiler-feedback presentation style affect programmers’ subjective user experience when the amount of technical guidance and explanatory content are controlled across conditions?

Pre-specified hypotheses

- **H1 (information-design effect):** Neutral AI explanations will receive higher pragmatic UEQ-S scores than raw compiler diagnostics.
- **H2 (persona effect):** Dr. Meowra will receive higher hedonic UEQ-S scores than neutral AI explanations.

H1 tests whether replacing conventional compiler output with a human-centered explanation improves the pragmatic experience of the feedback. H2 tests whether adding a persistent social persona provides an additional hedonic UX benefit beyond an otherwise comparable explanation.

Final feedback-style preference, study-specific Likert items, comprehension accuracy, and Agent Persona Instrument subscales will be treated as secondary or exploratory measures rather than additional confirmatory hypotheses.

3. Experimental Design

3.1 Design type

Within-subject design. Every participant experiences all three feedback conditions. This is efficient for a small one-month study because each participant serves as their own comparison, reducing noise from differences in programming experience.

Platform. A standalone Unity application is appropriate because it standardizes code presentation,

feedback formatting, response collection, and ordering. Participants are not asked to edit code. The study is about the experience of reading/interpreting feedback, not typing speed or IDE proficiency.

3.2 Six scenario baseline

Use six short C or C-like compiler-error scenarios: two per condition. Each scenario should contain one clear, local error and enough surrounding context to understand the diagnostic. Avoid obscure language features, cascading errors, or problems whose “correct” interpretation depends heavily on programmer intent.

Scenario family	Example error type	Reason to include
Statement termination	Missing semicolon	Familiar, local, easy to validate.
Name resolution	Undeclared identifier	Common diagnostic category.
Delimiter matching	Missing parenthesis or brace	Tests localization without obscure semantics.
Type use	Simple incompatible type	Adds semantic variety.
Function call	Wrong number/type of arguments	Common actionable diagnostic.
Operator use	Invalid operand/operator pairing	Adds another semantic form.

3.3 Illustrative stimulus

```
int main() {
    int total = 10
    printf("%d\n", total);
}
```

Condition	Illustrative feedback
Raw	error: expected "," or ";" before "printf"
Neutral AI	The compiler reached <code>printf</code> while it was still expecting the previous statement to end. Inspect the statement immediately above <code>printf</code> for a missing terminator.
Dr. Meowra	Dr. Meowra: The compiler reached <code>printf</code> while it was still waiting for the previous statement to finish. Take a look at the line just above <code>printf</code> for a missing terminator. Meowtastic! You are close! this just looks like a small syntax issue :)

Important: Before data collection, neutral/persona pairs should be reviewed for technical equivalence, similar length, similar amount of guidance, and absence of answer leakage.

4. Controlling the AI and Persona Manipulation

4.1 Persona introduction

Dr. Meowra will be presented as a persistent programming assistant rather than only as a writing style applied to individual error messages. Her visual appearance is shown in Figure 1.

Before a participant begins the Dr. Meowra condition, the application will provide a brief introduction establishing her name, role, visual identity, and supportive personality.



Figure 1: Dr. Meowra, the anthropomorphic programming assistant used in the persona-based compiler-feedback condition.

4.2 Dr. Meowra Persona Design

Dr. Meowra is presented as a catgirl programming assistant whose visual design incorporates feline *kemonomimi*. Kemonomimi refers to animal-like ears, such as dog or cat ears, depicted on humanoid characters [1]. In Dr. Meowra’s design, these features attempt to establish a visually recognizable character.

The use of a character-based interface is motivated by prior work on anthropomorphic and pedagogical agents. Lester et al. demonstrated a “persona effect,” in which the presence of a lifelike character positively affected users’ perceptions of an interactive learning experience, including dimensions such as helpfulness, credibility, and engagement [2]. Similarly, Moundridou and Virvou found that an anthropomorphic interface agent could improve participants’ experience of an intelligent tutoring system even when corresponding improvements in short-term learning outcomes were not observed [3].

Dr. Meowra is therefore not designed around a specific anime personality archetype such as *moe*. Instead, the character is designed to be competent, kind, encouraging, and nonjudgmental. Her visual identity and communication style are intended to establish social presence and make technical feedback feel more approachable while preserving the accuracy and technical content of the underlying explanation. Prior work further suggests that the manner in which pedagogical agents communicate including socially aware and polite feedback can influence users’ experience of the interaction [4].

4.3 Use one generation pipeline

Both the Neutral and Dr. Meowra conditions should be generated from the same underlying compiler output, source context, and model family. The persona condition should add social style, not additional technical assistance.

Constraint	Neutral AI	Dr. Meowra
Technical facts	Same	Same
Resolution hint	Same level	Same level
Approximate length	Matched	Matched
Tone	Professional, impersonal	Warm, encouraging, named persona, catgirl language
Avatar/name	None	Dr. Meowra identity; static avatar
Complete solution	Prohibited	Prohibited

Freeze all outputs before the experiment. Do not call a live LLM during participant sessions. Pre-generation removes model drift, stochastic differences, latency, service failures, and accidental differences in assistance. Save the model, prompt, AI.md/persona file, and final stimuli for reproducibility.

4.4 Suggested AI.md specification

- Accurate and concise: explain only what is supported by the compiler message and code context.
- No complete corrected program and no direct answer to the comprehension question.
- Explain what the compiler encountered/expected and where the programmer should inspect.

- Warm, nonjudgmental, encouraging tone; never shame the programmer.
- Preserve technical terminology when useful, but explain it in accessible language.
- Persona markers should be subtle enough that technical content remains primary.

4.5 Persona introduction and relational continuity

Dr. Meowra will be presented as a persistent programming assistant rather than only as a writing style applied to individual error messages. Before a participant begins the Dr. Meowra condition, the application will provide a brief introduction establishing her name, role, visual identity, and supportive personality.

For example, Dr. Meowra may briefly introduce herself as a programming assistant whose role is to help explain compiler feedback. This introduction is intended to establish familiarity and social presence before participants evaluate her feedback.

The persona introduction is considered part of the Dr. Meowra treatment. Therefore, the general experiment tutorial will remain condition-neutral and will teach only the mechanics of reading a scenario, selecting a response, and advancing through the application.

Because condition order is counterbalanced, the Dr. Meowra introduction will appear immediately before the Dr. Meowra block regardless of whether that block occurs first, second, or third.

Construct validity decision

Since Dr. Meowra includes a visible catgirl avatar plus named/social wording, the experiment tests the persona as a package. We can not claim that tone alone caused the effect. A later factorial study can separate avatar, name, first-person language, warmth, and other anthropomorphic cues.

5. Participant Flow

1. Consent and study instructions.
2. Brief demographics and programming-background questionnaire (~5 items).
3. Condition-neutral Unity tutorial explaining how to read a scenario, select a multiple-choice response, and advance through the experiment.
4. Short practice scenario to verify that the participant understands the interface.
5. Condition Block 1: two compiler scenarios, each followed by one lightweight comprehension item.
6. Condition Evaluation 1: UEQ-S and four targeted 7-point agreement items.
7. Condition Block 2 followed by Condition Evaluation 2.
8. Condition Block 3 followed by Condition Evaluation 3.
9. Immediately before the Dr. Meowra block, regardless of its position in the counterbalanced sequence, present the brief Dr. Meowra persona introduction.
10. After all three conditions, administer the selected Credible and Engaging subscales of the Agent

Persona Instrument for Dr. Meowra.

11. Final forced-choice preference among Raw, Neutral, and Dr. Meowra.
12. One open-ended question: “Briefly explain why you preferred that feedback style.”
13. Debrief.

Conditional persona introduction. The Dr. Meowra introduction is inserted immediately before whichever numbered condition block has been assigned to the Meowra condition. It is therefore not a fixed chronological step in the sequence above.

Estimated duration. Roughly 12–18 minutes with six scenarios, depending on reading speed and the number of persona items retained. Pilot this rather than guessing.

6. Measures

6.1 Primary: UEQ-S after each condition

The UEQ-S (Schrepp et al., 2017) is an eight-item short user-experience measure. It yields pragmatic quality, hedonic quality, and an overall UX score. It uses 7-position semantic-differential items, not “seven separate questionnaires.” The same UEQ-S is administered once after Raw, once after Neutral, and once after Dr. Meowra.

Score	Interpretation in this study	Expected pattern (hypothesis, not assumption)
Pragmatic quality	How supportive, easy, efficient, and clear the feedback experience feels.	Neutral and Meowra may exceed Raw.
Hedonic quality	How interesting, exciting, inventive, and novel the experience feels.	Meowra may exceed Neutral and Raw.
Overall UX	Compact summary across all eight items.	Exploratory overall comparison.

Interpretation caution. Hedonic quality includes novelty/inventiveness. A catgirl persona is intentionally novel, so a hedonic advantage may partly reflect novelty. That is a limitation to discuss, not a reason to avoid the scale.

6.2 Four targeted 7-point agreement items

After each condition, ask the same four study-specific statements on a 1 = strongly disagree to 7 = strongly agree scale. These are direct measures, not claimed as a validated standalone instrument:

- “I found this feedback helpful.”
- “I would trust this feedback while programming.”
- “This feedback would make compiler errors feel less frustrating.”
- “I would like my programming tools to communicate errors in this style.”

Why keep these if UEQ-S exists? UEQ-S gives a validated general UX measure. These four items capture constructs that are specifically meaningful for AI programming feedback: helpfulness,

trust, frustration, and adoption preference.

Analysis role. These study-specific items are secondary measures intended to help interpret the UEQ-S results. They will primarily be summarized descriptively across conditions. Individual Likert items will not be treated as separate confirmatory hypotheses. Any inferential tests involving these items will be explicitly identified as exploratory and corrected for multiple comparisons where appropriate.

6.3 Dr. Meowra characterization with the Agent Persona Instrument

Ryu and Baylor (2005) validated the Agent Persona Instrument around Credible, Facilitating Learning, Human-like, and Engaging factors. The published instrument uses a 5-point Likert response format. It is useful here as a secondary manipulation/characterization check: did participants actually perceive Dr. Meowra as credible and engaging?

Recommended shortening

Since Dr. Meowra is a static-avatar we will use the Credible and Engaging subscales (5 items each) rather than the full 25-item API, and report explicitly that selected subscales were administered. Several Human-like items refer to movement/emotion and are poorly matched to a static text interface.

Do not convert the API to 7 points merely for visual consistency. If using published API subscales, preserve the original 5-point response format unless you have a strong reason and clearly disclose the modification.

6.4 Lightweight comprehension check

One multiple-choice item follows each scenario. This is not a full debugging test. Its purpose is to verify that participants are reading the message and to detect any obvious trade-off where a more pleasant style produces worse understanding.

- Example form: “Which line would you inspect first based on this feedback?”
- Alternative: “Which explanation best matches the compiler problem?”
- Score: correct/incorrect; report as secondary/exploratory.

The rationale for not requiring code editing. Typing a fix introduces additional constructs - coding speed, syntax recall, IDE familiarity, debugging strategy, and trial-and-error behavior. Because the primary construct is UX of diagnostic feedback, controlled reading/interpretation tasks provide a cleaner exposure to the treatment. Ecological validity should be acknowledged as a limitation and direct debugging can be future work.

6.5 Final preference and qualitative response

- Forced choice: “If you could choose one of these feedback styles for your programming environment, which would you choose?”
- Open response: “Briefly explain why.”

This gives the paper a simple headline preference result while the open response can reveal themes such as reassurance, distraction, clarity, professionalism, novelty, annoyance, or trust.

7. Data Collection and Logging

The Unity application will automatically record study responses using an anonymous participant identifier. Participant names are not required by the experimental application.

For each participant, the application will record:

- anonymous participant ID;
- assigned condition order;
- assigned scenario/stimulus set;
- scenario ID;
- feedback condition;
- selected comprehension response;
- comprehension correctness;
- response time for each scenario;
- UEQ-S item responses;
- targeted 7-point agreement responses;
- selected Agent Persona Instrument subscale responses;
- final preferred feedback style; and
- optional open-ended explanation.

Response-time data will be recorded automatically as an exploratory measure but will not constitute a primary dependent variable.

The application will save participant data incrementally after completed responses rather than storing the entire session only at the end. This reduces the risk of losing a complete participant session following an application or system failure.

Experimental data will be exported in a structured format suitable for later analysis, such as CSV or JSON. The final storage location and retention procedure will follow the requirements of the approved institutional human-subjects protocol.

8. Counterbalancing and Assignment

Do not let every participant experience Raw → Neutral → Meowra. Condition order can create practice, fatigue, contrast, and novelty effects. Use all six possible block orders or a balanced subset:

Order	Sequence
1	Raw → Neutral → Meowra
2	Raw → Meowra → Neutral
3	Neutral → Raw → Meowra
4	Neutral → Meowra → Raw
5	Meowra → Raw → Neutral
6	Meowra → Neutral → Raw

Also rotate scenario-to-condition assignment. Otherwise a particularly easy error could make one condition look better. With six scenarios, create at least three stimulus sets so that each error family appears under different feedback styles across participants.

Minimum practical rule

Counterbalance both (1) which condition appears first and (2) which specific compiler errors appear in each condition. This matters more for internal validity than adding another ten questionnaire items.

9. Participants and Sample Size

Recruit participants who can read small C/C-like programs and have at least introductory programming exposure. Undergraduate CS/CSE students are a reasonable population if the claims are explicitly about student/novice-to-intermediate programmers. Record programming experience so it can be described and, if necessary, explored as a moderator.

Pragmatic target. The study will target approximately 18 completed participants, with a feasible range of 15–20 participants. Because the design is within-subject, each participant contributes observations to all three feedback conditions and therefore serves as their own control.

The target of 18 participants is also convenient for counterbalancing: the six possible condition orders can each be assigned to three participants. When combined with three scenario-to-condition assignment sets, 18 participants allow the study to distribute participants across the resulting counterbalancing combinations evenly.

Given this small exploratory sample, the study is expected to be most sensitive to relatively large within-participant effects. A sensitivity analysis will be conducted before data collection to characterize the magnitude of effect that can be detected with the final recruitment target. Statistical significance will therefore be interpreted alongside effect-size estimates rather than as the sole evidence for or against the intervention.

Useful background variables (keep short):

- Years of programming experience (categorical bands).
- Self-rated C/C++ familiarity.
- Frequency of programming.
- Prior use of AI coding assistants.
- Student level / relevant coursework, if needed for sample description.

10. Analysis Plan

The analysis will focus on two planned within-participant comparisons that correspond directly to the experimental design:

1. **Raw vs. Neutral on Pragmatic Quality:** tests whether replacing a conventional compiler diagnostic with a human-centered explanation improves the pragmatic user experience.
2. **Neutral vs. Dr. Meowra on Hedonic Quality:** tests whether adding a social persona improves the hedonic user experience beyond an otherwise comparable human-centered explanation.

The second comparison is the primary comparison of theoretical interest because it most directly evaluates the contribution of persona-based presentation while approximately controlling the explanatory information provided.

A direct Raw vs. Dr. Meowra significance test will not be treated as a confirmatory comparison. Those conditions differ in both explanatory content and persona framing, making it difficult to attribute any observed difference specifically to the persona.

Outcome	Analysis	Report
UEQ-S Pragmatic Quality	Confirmatory test for H1. A paired Wilcoxon signed-rank test will compare Raw and Neutral pragmatic scores.	Condition medians and interquartile ranges, Wilcoxon statistic, Holm-adjusted p -value, and paired effect size such as rank-biserial correlation.
UEQ-S Hedonic Quality	Confirmatory test for H2. A paired Wilcoxon signed-rank test will compare Neutral and Dr. Meowra hedonic scores.	Condition medians and interquartile ranges, Wilcoxon statistic, Holm-adjusted p -value, and paired effect size such as rank-biserial correlation.
UEQ-S Overall Score	Secondary descriptive outcome. Overall UX scores will be summarized across all three conditions but will not constitute an additional confirmatory hypothesis.	Condition summaries and distributions.
Targeted 7-point items	Secondary measures. Helpfulness, trust, frustration, and willingness-to-use ratings will primarily be summarized descriptively. Any inferential analysis will be explicitly identified as exploratory.	Condition medians, distributions, and paired differences where useful.
Comprehension accuracy	Secondary exploratory measure. With only two scenarios per condition, accuracy will primarily be used to identify obvious comprehension trade-offs rather than support strong performance claims.	Correct responses and percentage accuracy by condition.
Final preference	Descriptive comparison of participants' final preferred feedback style.	Counts and percentages for Raw, Neutral, and Dr. Meowra.
Open-ended explanation	Lightweight qualitative coding of recurring reasons for participant preferences.	Theme frequencies and short representative quotations or paraphrases where permitted.

10.1 Confirmatory Statistical Tests

Both primary hypotheses concern paired observations because every participant experiences every condition.

For H1, the within-participant difference is:

$$D_i^{(P)} = \text{Neutral Pragmatic Score}_i - \text{Raw Pragmatic Score}_i.$$

For H2, the within-participant difference is:

$$D_i^{(H)} = \text{Meowra Hedonic Score}_i - \text{Neutral Hedonic Score}_i.$$

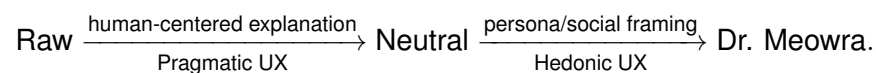
A Wilcoxon signed-rank test will be used for each comparison. The test evaluates whether the direction and relative magnitude of the paired differences systematically favor one condition rather than being centered around no difference.

Because two confirmatory statistical tests are performed, Holm correction will be applied across the two resulting p -values to control the family-wise Type I error rate.

Statistical significance will not be interpreted in isolation. Rank-biserial correlations will be reported as paired effect-size estimates so that the magnitude and consistency of the observed treatment effects can be evaluated. This is particularly important given the expected sample size of approximately 15–20 participants.

10.2 Interpretation

The two confirmatory comparisons correspond to two conceptually different changes in the interface:



A significant Raw vs. Neutral pragmatic difference would support the claim that human-centered explanation improves the practical experience of compiler feedback.

A significant Neutral vs. Dr. Meowra hedonic difference would support the claim that persona-based presentation provides an additional experiential benefit beyond the explanatory information itself.

Failure to obtain statistical significance will not be interpreted as evidence that the conditions are equivalent. Given the small exploratory sample, effect sizes, score distributions, participant preference, and uncertainty will also be considered when interpreting the findings.

11. Threats to Validity to Plan for Now

Threat	Mitigation / interpretation
Novelty effect	A catgirl assistant is unusual. Higher hedonic ratings may not persist with long-term use. State this directly; longitudinal use is future work.
Message equivalence	If Meowra accidentally gives more precise guidance, the persona effect is confounded with information quality. Match content and have independent reviewers check pairs.
Demand characteristics	Participants may infer that the study wants them to like the playful condition. Neutral study wording and counterbalancing reduce this risk.
Ecological validity	A Unity reading task is not a real IDE session. This is acceptable for isolating UX, but it limits claims about real debugging performance.
Population	Student programmers may respond differently from experienced professionals. Match claims to the sampled population.
Repeated measures	Seeing three styles can create contrast effects. Counterbalance order and avoid reusing the exact same code problem across conditions for a participant.

12. Candidate Contribution Statement

Paper-level claim

We present a controlled within-subject study of persona-based AI compiler feedback. By comparing raw compiler diagnostics, neutral AI explanations, and technically matched explanations delivered through a friendly named persona, we examine whether social framing changes the user experience of programming diagnostics. Our goal is not to establish that persona improves debugging performance, but to isolate UX as a design dimension for AI-assisted programming tools.

This scope leaves a coherent future-work program: direct debugging performance, learning/retention, cognitive load, professional developers, longitudinal habituation, adaptive personas, user-selectable personas, and factorial studies separating avatar, wording, name, encouragement, and anthropomorphic cues.

13. Pass One: What Each Source Gives You

Role in paper	Source	Why it matters
Compiler diagnostics as HCI	Traver (2010)	Foundational framing; explicitly anticipates affective computing and more reassuring compiler behavior.

Role in paper	Source	Why it matters
Landscape of error-message research	Becker et al. (2019)	Broad review and design-guideline map; establishes that programming diagnostics are a mature but unresolved empirical area.
Enhanced messages can help	Becker (2016); Denny et al. (2020)	Evidence that message design/readability can affect errors, debugging time, and perceived usefulness.
Enhanced messages do not always help	Denny et al. (2014)	Important counterweight; prevents simplistic “more explanation is always better” framing.
Presentation itself matters	Dong & Khandwala (2019)	Shows that seemingly cosmetic presentation changes can affect error-message usability/comprehension.
Developers actually read diagnostics	Barik et al. (2017)	Eye-tracking evidence that error-message reading is cognitively meaningful and predicts task performance.
Explanation structure matters	Barik et al. (2018)	Provides design theory for why explanation quality/structure affects developer preference.
LLM error explanations	Taylor et al. (2024)	Demonstrates scalable context-aware LLM compiler explanations.
LLMs are not automatically superior	Santos & Becker (2024)	Directly motivates studying UX/persona rather than assuming AI-generated wording improves debugging performance.
Conversational AI debugging	Renzella et al. (2025)	Shows current movement from static explanations toward conversational compiler-integrated AI.
Computers as social actors	Nass et al. (1994)	Classic theoretical basis for social responses to computational systems.
Persona effect	Lester et al. (1997)	Direct evidence that lifelike pedagogical agents can affect perceived learning experience.
Social agency	Moreno et al. (2001)	Tests identical instructional content delivered with agent/social features; conceptually close to matched Neutral-vs.-Persona design.
Persona measurement	Ryu & Baylor (2005)	Validated Agent Persona Instrument; supports credibility/engagement characterization.
Anthropomorphism in modern LLMs	Cohn et al. (2024)	Contemporary evidence that anthropomorphic cues can change perceptions of LLM systems.

Role in paper	Source	Why it matters
Schrepp et al. (2017)	Validation and design of UEQ-S; supports pragmatic/hedonic UX outcomes.	

References

- [1] R. Shijo, S. Sakurai, K. Hirota, and T. Nojima, "Consideration of emotional expression interface based on emotional expression ability of animal," *The Transactions of Human Interface Society*, vol. 23, no. 4, pp. 419–430, 2021.
- [2] J. C. Lester, S. A. Converse, S. E. Kahler, S. T. Barlow, B. A. Stone, and R. S. Bhogal, "The persona effect: Affective impact of animated pedagogical agents," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 1997.
- [3] M. Moundridou and M. Virvou, "Evaluating the persona effect of an interface agent in a tutoring system," *Journal of Computer Assisted Learning*, vol. 18, no. 3, pp. 253–261, 2002.
- [4] N. Wang, W. L. Johnson, R. E. Mayer, P. Rizzo, E. Shaw, and H. Collins, "The politeness effect: Pedagogical agents and learning outcomes," *International Journal of Human-Computer Studies*, vol. 66, no. 2, pp. 98–112, 2008.
- [5] T. Barik, D. Ford, E. Murphy-Hill, and C. Parnin, "How should compilers explain problems to developers?" in *Proceedings of the 26th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2018, pp. 633–643.
- [6] T. Barik, J. Smith, K. Lubick, E. Holmes, J. Feng, E. Murphy-Hill, and C. Parnin, "Do developers read compiler error messages?" in *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*, 2017, pp. 575–585.
- [7] B. A. Becker, "An effective approach to enhancing compiler error messages," in *Proceedings of the 47th ACM Technical Symposium on Computing Science Education*, 2016, pp. 126–131.
- [8] B. A. Becker, P. Denny, R. Pettit, D. Bouchard, D. J. Bouvier, B. Harrington, A. Kamil, A. Karkare, C. McDonald, P.-M. Osera, J. L. Pearce, and J. Prather, "Compiler error messages considered unhelpful: The landscape of text-based programming error message research," in *2019 ITiCSE Working Group Reports*, 2019, pp. 177–210.
- [9] M. Cohn, M. Pushkarna, G. O. Olanubi, J. M. Moran, D. Padgett, Z. Mengesha, and C. Heldereth, "Believing anthropomorphism: Examining the role of anthropomorphic cues on trust in large language models," in *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, 2024.
- [10] P. Denny, A. Luxton-Reilly, and D. Carpenter, "Enhancing syntax error messages appears ineffectual," in *Proceedings of the 2014 Conference on Innovation & Technology in Computer Science Education*, 2014, pp. 273–278.
- [11] P. Denny, J. Prather, and B. A. Becker, "Error message readability and novice debugging performance," in *Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education*, 2020, pp. 480–486.
- [12] T. Dong and K. Khandwala, "The impact of "cosmetic" changes on the usability of error mes-

- sages,” in *Extended Abstracts of the 2019 CHI Conference on Human Factors in Computing Systems*, 2019.
- [13] R. Moreno, R. E. Mayer, H. A. Spires, and J. C. Lester, “The case for social agency in computer-based teaching: Do students learn more deeply when they interact with animated pedagogical agents?” *Cognition and Instruction*, vol. 19, no. 2, pp. 177–213, 2001.
 - [14] C. Nass, J. Steuer, and E. R. Tauber, “Computers are social actors,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 1994, pp. 72–78.
 - [15] J. Renzella, A. Vassar, L. Lee Solano, and A. Taylor, “Compiler-integrated, conversational ai for debugging cs1 programs,” in *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1*, 2025, pp. 994–1000.
 - [16] J. Ryu and A. L. Baylor, “The psychometric structure of pedagogical agent persona,” *Technology, Instruction, Cognition & Learning*, vol. 2, no. 4, pp. 291–315, 2005. [Online]. Available: <https://amybaylor.com/Articles/2005TICL.pdf>
 - [17] E. A. Santos and B. A. Becker, “Not the silver bullet: Llm-enhanced programming error messages are ineffective in practice,” in *Proceedings of the 2024 Conference on United Kingdom & Ireland Computing Education Research*, 2024, pp. 1–7.
 - [18] M. Schrepp, A. Hinderks, and J. Thomaschewski, “Design and evaluation of a short version of the user experience questionnaire (ueq-s),” *International Journal of Interactive Multimedia and Artificial Intelligence*, vol. 4, no. 6, pp. 103–108, 2017.